PYTHON DATA APPS

Plotly, Dash & Other Tools

From a Plotly chart to an interactive app, what happens behind the scenes, and the other Python app tools

Agenda

1. Plotly vs Dash: why a server 5 min
2. Building a page: layout, charts, widgets 7 min
3. Callbacks, and clicking one chart to update another 8 min
4. Letting Copilot write Dash for you 4 min
5. Showcase, and other tools 5 min

1 · PLOTLY ALONE

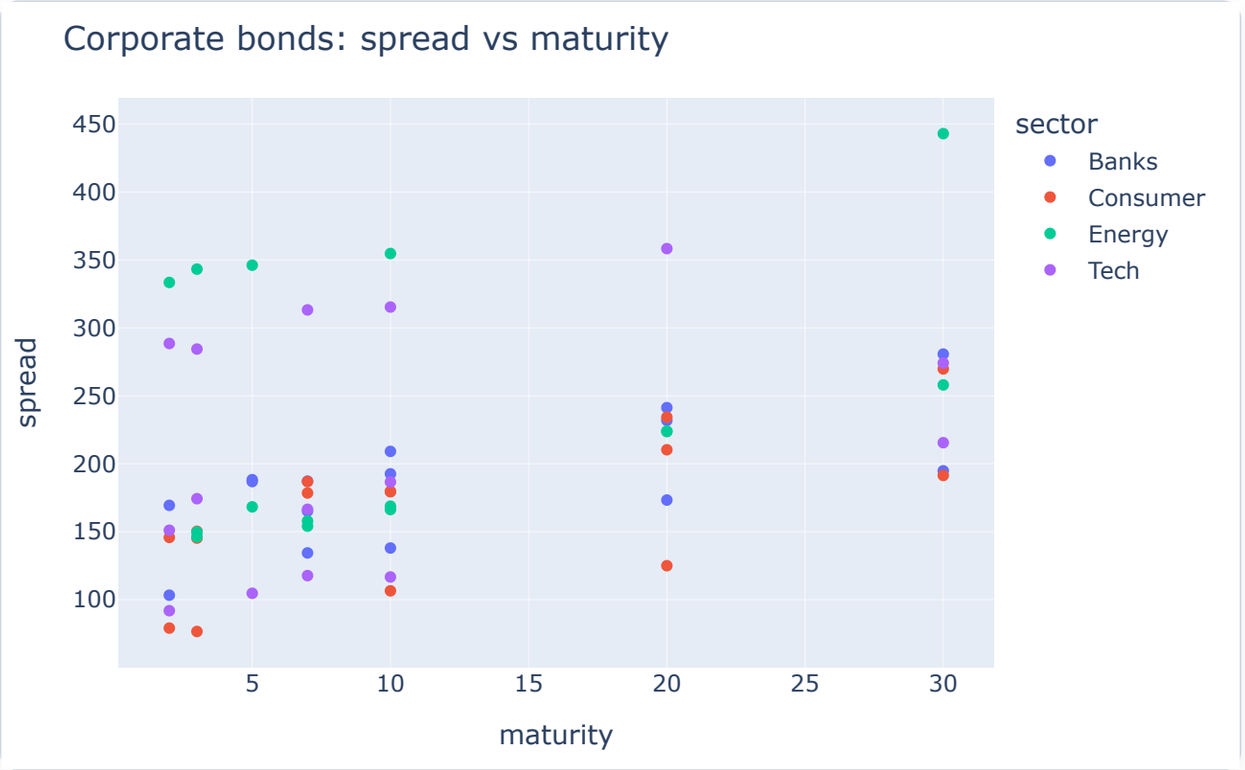
Plotly: interactive charts, no server

```
import plotly.express as px
from data import bonds

fig = px.scatter(bonds, x="maturity", y="spread",
                 color="sector", hover_name="issuer")
fig.write_html("plotly_only.html")
```

Needs Python again

- recomputation
- call a model, run a pricer
- data too big to ship into one HTML file / query a database

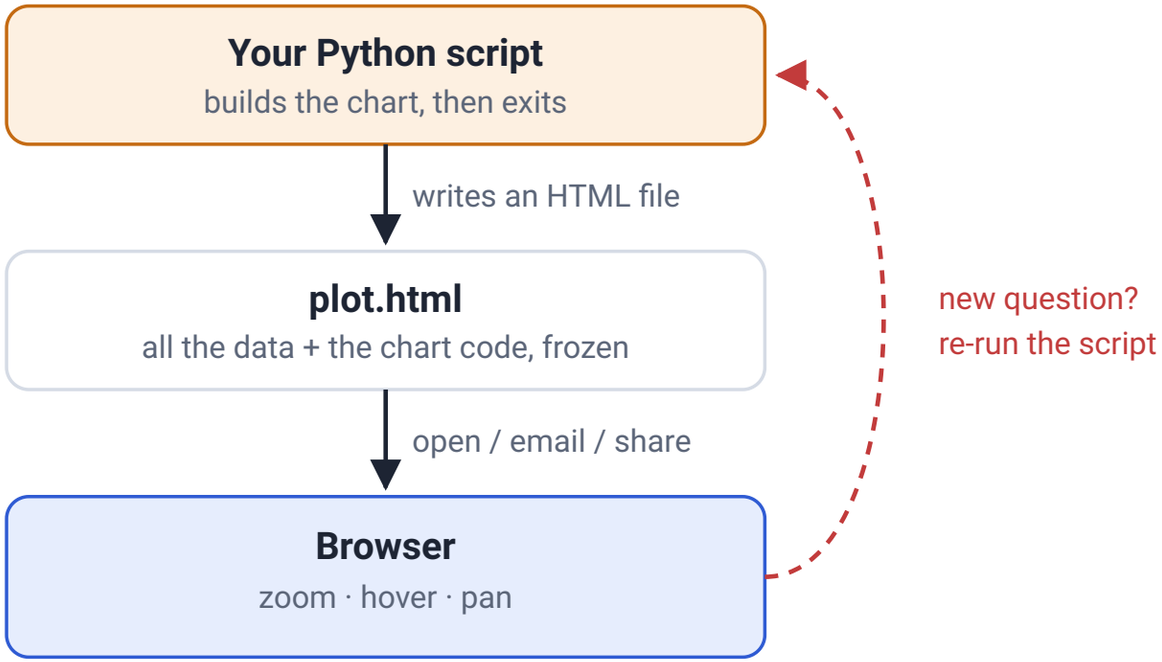


Works in the browser: zoom, hover, legend.

Plotly has no back end. Once the HTML is written, nothing can ask Python a new question.

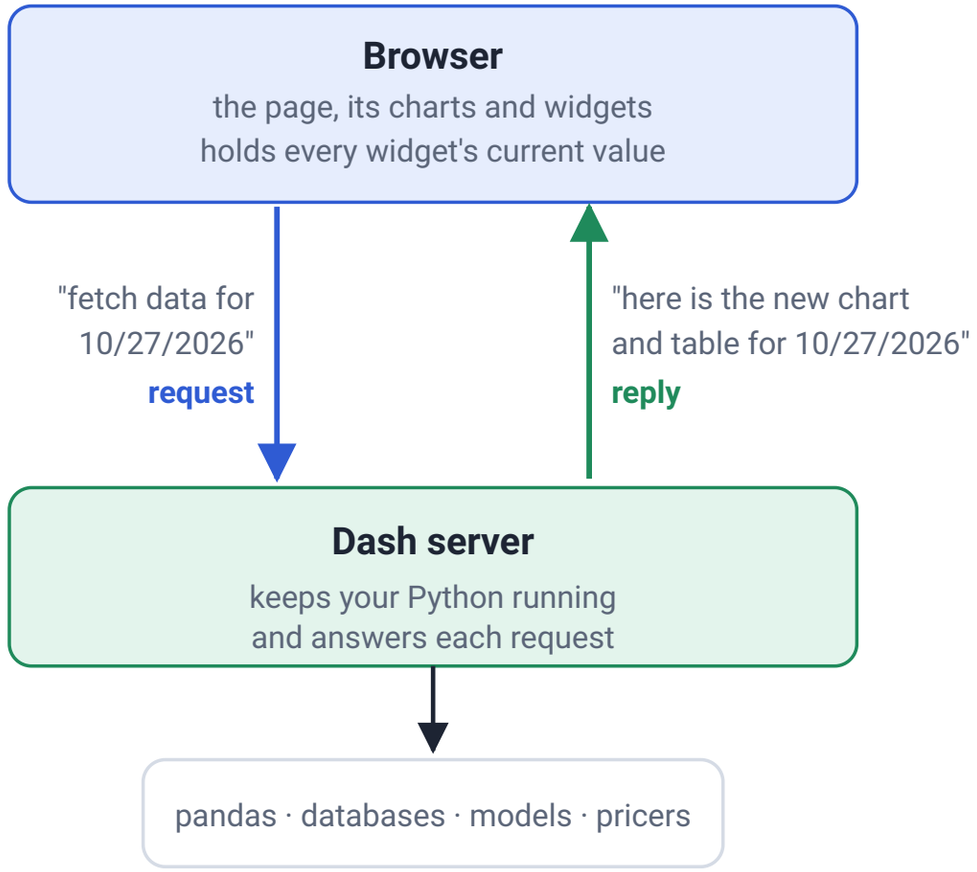
Plotly vs Dash

Plotly alone: one way, once



No server: nothing is listening after the file is written.

Dash: a conversation



The smallest Dash app

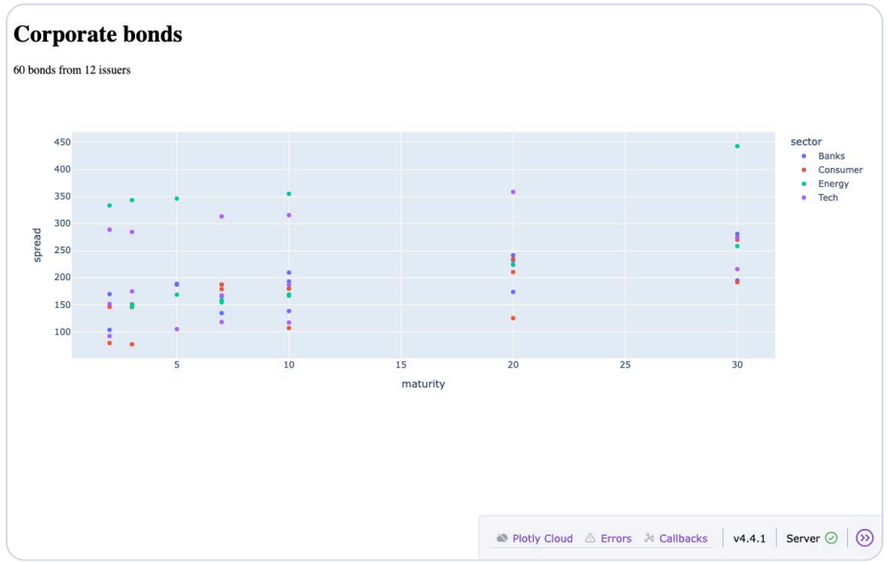
```python
from dash import Dash, html, dcc
import plotly.express as px
from data import bonds

app = Dash(__name__)

fig = px.scatter(bonds, x="maturity", y="spread", color="sector")

app.layout = html.Div([
    html.H1("Corporate bonds"),
    html.P(f"{len(bonds)} bonds"),
    dcc.Graph(id="chart", figure=fig),  # id: find it later
])

if __name__ == "__main__":
    app.run(debug=True)     # http://127.0.0.1:8050
```



demos/02_hello_dash.py

Components nest like boxes: the order in the code is the order on the page. An `id` lets the app find a component later, to read it or update it.

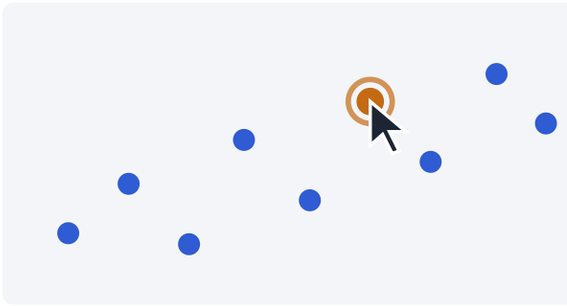
Plotly charts inside Dash

`dcc.Graph` shows any Plotly chart: the same code you'd write in a notebook.

```
dcc.Graph(
    id="chart",
    figure=px.line(df, x="date",
                   y="spread"),
)
```

A chart also tells the app what the user does to it.

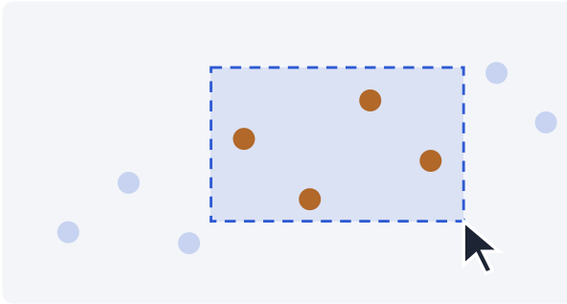
Later we'll show how to use `clickData` to update another chart when a point is clicked.



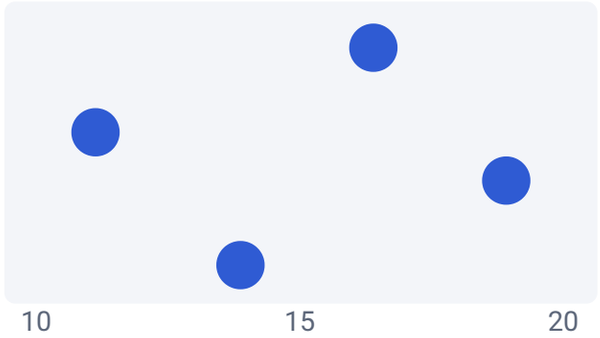
Click a point: the app learns which point
`clickData`



Hover a point: the app learns which point is under the mouse
`hoverData`

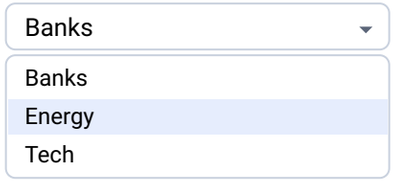


Draw a box or lasso: the app learns which points are inside
`selectedData`



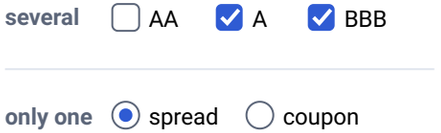
Zoom or pan: the app learns the new visible range
`relayoutData`

Widgets you get for free



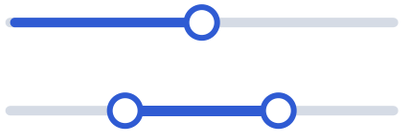
Pick from a list: the app gets the chosen item (or several)

`dcc.Dropdown`



Checklist: tick several. Radio: exactly one. The app gets what is selected

`dcc.Checklist` · `dcc.RadioItems`



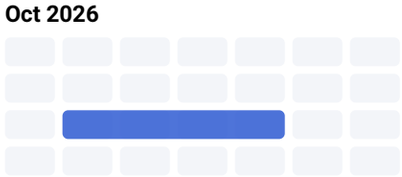
Drag a handle: the app gets the number, or the range

`dcc.Slider` · `dcc.RangeSlider`



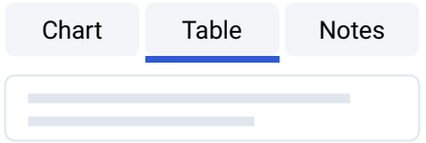
Type text or a number: the app gets what was typed

`dcc.Input`



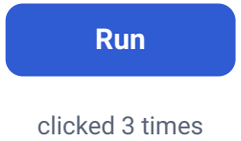
Pick a date or a range: the app gets the dates

`dcc.DatePickerSingle` · `DatePickerRange`



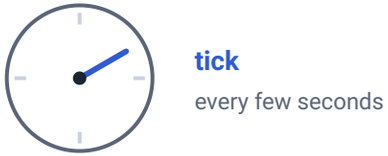
Switch tabs: the app gets which tab is open

`dcc.Tabs`



Press a button: the app gets how many times it was clicked

`html.Button`



A timer: the app gets a tick every few seconds, for live updates

`dcc.Interval`



Keep data between steps; let files in and out

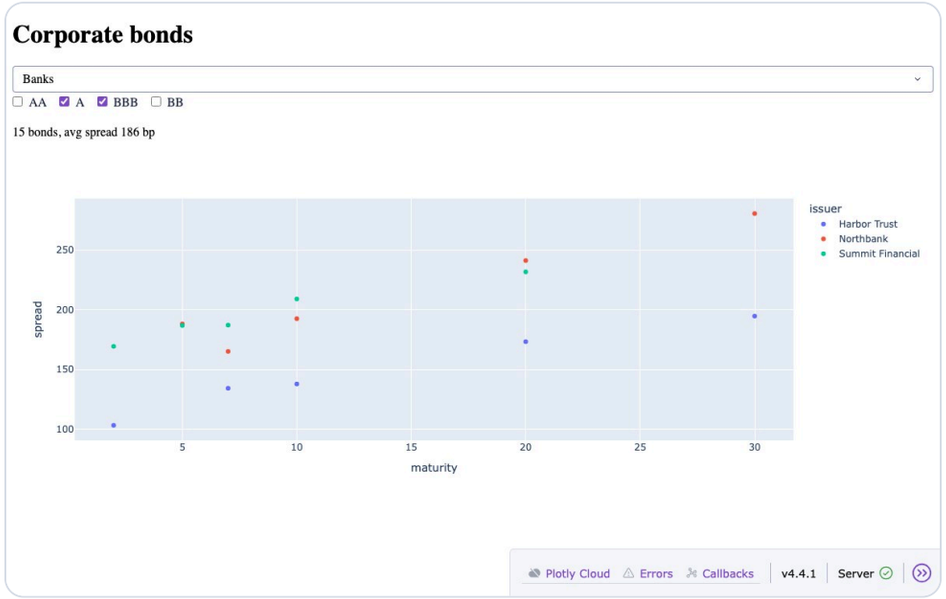
`dcc.Store` · `dcc.Upload` · `dcc.Download`

Add-on libraries: `dash-ag-grid` (sortable, filterable tables) · `dash-bootstrap-components` and `dash-mantine-components` (themes, layouts, more widgets)

A callback: "when this changes, run that"

```
app.layout = html.Div([
    html.H1("Corporate bonds"),
    dcc.Dropdown(sectors, "Banks", id="sector"),
    dcc.Checklist(ratings, ["A", "BBB"], id="rating"),
    html.P(id="summary"),
    dcc.Graph(id="chart"),
])

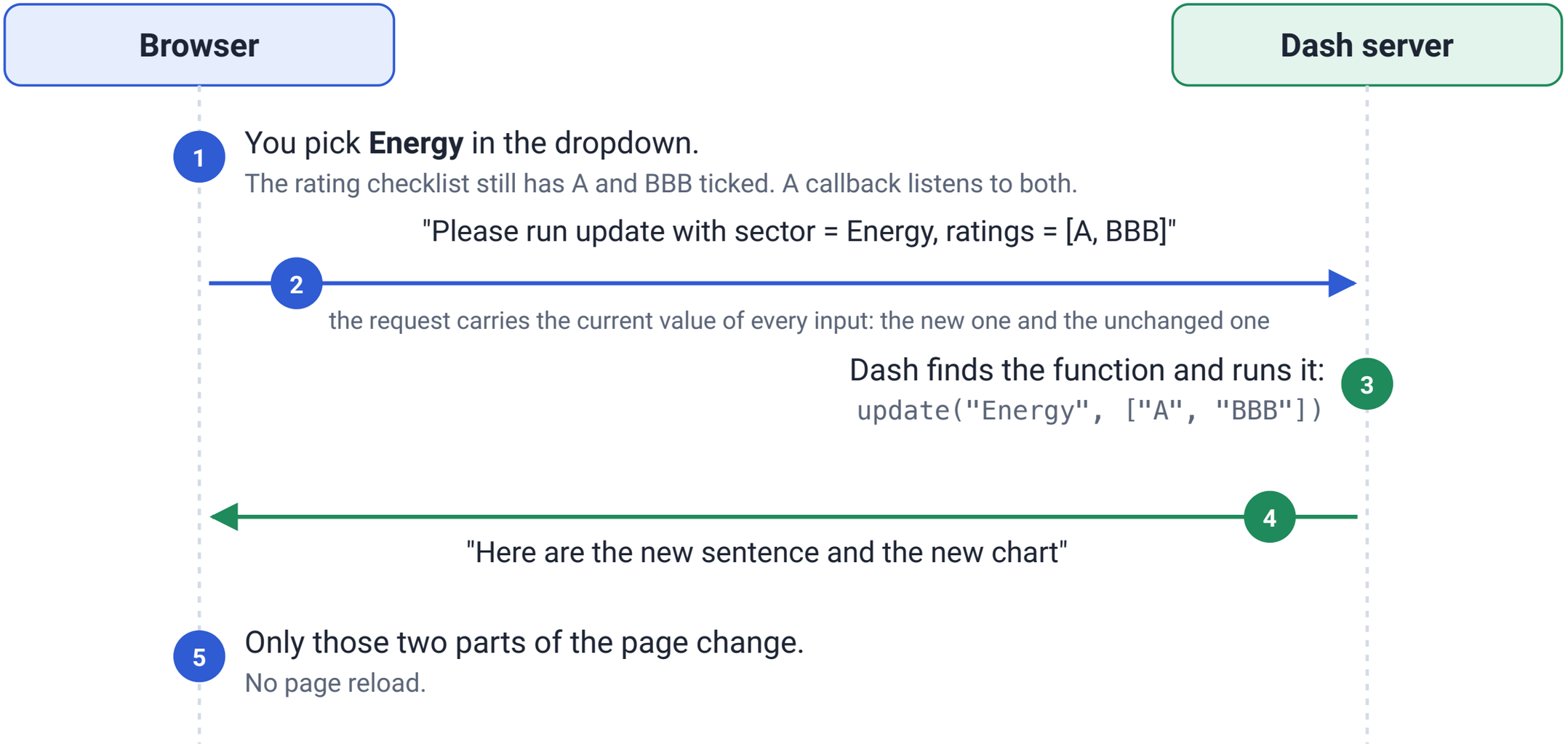
@callback(Output("summary", "children"),
          Output("chart", "figure"),
          Input("sector", "value"),
          Input("rating", "value"))
def update(sector, ratings):
    d = bonds[(bonds.sector == sector)
              & bonds.rating.isin(ratings)]
    text = f"{len(d)} bonds, avg {d.spread.mean():.0f} bp"
    fig = px.scatter(d, x="maturity", y="spread")
    return text, fig
```



Inputs → arguments, in order. The first `Input` arrives as the first argument (`sector`), the second as the second (`ratings`).

Return values → Outputs, in order. `text` fills the first `Output`, `fig` the second.

How the browser hands the work back to Python



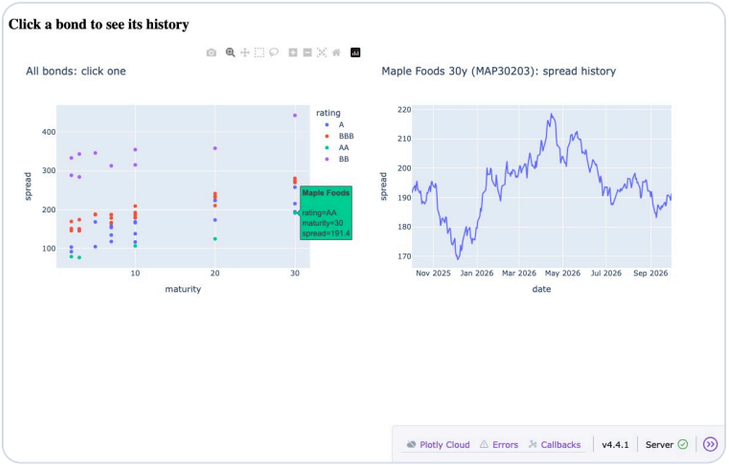
3 · CLICK ONE CHART, UPDATE ANOTHER

Click a bond, see its history

```
overview = px.scatter(bonds, x="maturity", y="spread",
                      color="rating", hover_name="issuer",
                      custom_data=["cusip"])   # rides along with each point

app.layout = html.Div([
    dcc.Graph(id="overview", figure=overview),
    dcc.Graph(id="detail"),
])

@callback(Output("detail", "figure"),
          Input("overview", "clickData"))
def show_history(click):
    if click is None:                       # page load: nothing clicked
        cusip = bonds.cusip.iloc[0]
    else:
        cusip = click["points"][0]["customdata"][0]
    h = history[history.cusip == cusip]
    return px.line(h, x="date", y="spread", title=cusip)
```



demos/05_click_detail.py

4 · VS CODE + COPILOT

Let Copilot write the Dash code

Describe what you want: at a high level, or in more detail if you know the details.

High level

Build a Dash app for the bonds data: filter by sector, show spread vs maturity, and let me click a bond to see its history.

More detailed

Add a sector dropdown (id `sector`, default Banks). When it changes, filter the scatter and show the count and average spread under it. When I click a point, use `custom_data` to get the cusip and plot its history from `history` in a second chart.

Tip: write your rules down once (a Copilot instructions file or a skill): which Dash version to use, where the data comes from, your naming conventions. Copilot then follows them on every request.

5 · SHOWCASE

Showcase: a real dashboard

Switching to the dashboard now.

5 · OTHER TOOLS

Dash and other Python app tools

tool	best fit	front-end model	server
Dash	production dashboards and apps with clear interactivity; Plotly charts	the page is described in Python; the browser holds widget values and asks the server when an input changes	Python web server; each request is answered on its own, with no per-user memory
Streamlit	quick internal tools; turning a data script into an app fast	the whole script re-runs top to bottom on every interaction; widgets simply return values	Python server with a live connection per user; per-user state in `st.session_state`
Panel / Bokeh	notebook-to-app; scientific or large data; mixing plotting libraries	Python objects kept in sync with their browser copies; reactive functions	Bokeh server with a live connection per user; can also run inside Jupyter
Jupyter widgets	interactive exploration inside a notebook	widgets in the notebook, kept in sync with Python objects	the notebook's Python kernel; Voilà turns a notebook into a shareable page
Shiny for Python	apps with many linked inputs and outputs	reactive: outputs re-compute automatically when the inputs they use change	Python server with a live connection per user
Gradio	demos of ML models: inputs in, prediction out	a function wrapped in ready-made input and output components	Python server; easy to share or host on Hugging Face

APPENDIX

Component code

One small, runnable app per card on slides 6 and 7, plus live tick data with `dcc.Interval`. Click a card in the talk to open its slide; ← **Back** returns to where you were.

1. `dcc.Graph · clickData`
2. `dcc.Graph · hoverData`
3. `dcc.Graph · selectedData`
4. `dcc.Graph · relayoutData`
5. `dcc.Dropdown`
6. `dcc.Checklist · dcc.RadioItems`
7. `dcc.Slider · dcc.RangeSlider`
8. `dcc.Input`
9. `dcc.DatePickerSingle · DatePickerRange`
10. `dcc.Tabs`
11. `html.Button`
12. `dcc.Interval`
13. `dcc.Interval · live tick data`
14. `dcc.Store · dcc.Upload · dcc.Download`

COMPONENT CODE

dcc.Graph · clickData

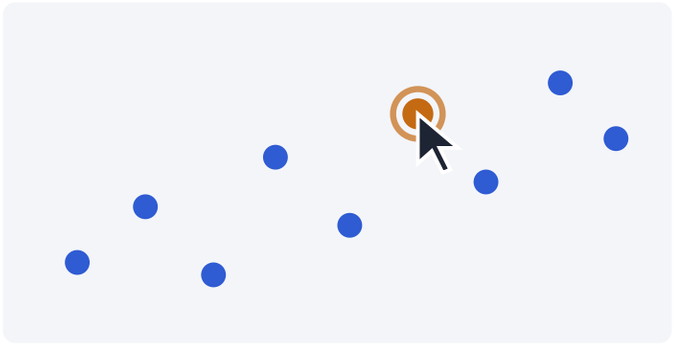
```python
"""dcc.Graph clickData: click a point and the app learns which point."""
from dash import Dash, html, dcc, Input, Output
import plotly.express as px

fig = px.scatter(x=[2, 3, 5, 7, 10, 20, 30],          # maturity
                 y=[80, 95, 120, 140, 165, 210, 240])  # spread

app = Dash(__name__)
app.layout = html.Div([
    dcc.Graph(
        id="chart",
        figure=fig,
        clickData=None,           # filled in when a point is clicked
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("chart", "clickData"))
def show(click):
    if click is None:                     # nothing clicked yet
        return "Click a point"
    p = click["points"][0]                # the clicked point
    return f"maturity {p['x']}, spread {p['y']}"

if __name__ == "__main__":
    app.run(debug=True)
```



Click a point: the app learns which point

1 the component 2 the callback that reads its value
(→ moves from 1 to 2)

Try it: python demos/components/g1_click.py

COMPONENT CODE

dcc.Graph · hoverData

```python
"""dcc.Graph hoverData: which point is under the mouse."""
from dash import Dash, html, dcc, Input, Output
import plotly.express as px

fig = px.scatter(x=[2, 3, 5, 7, 10, 20, 30],          # maturity
                 y=[80, 95, 120, 140, 165, 210, 240])  # spread

app = Dash(__name__)
app.layout = html.Div([
    dcc.Graph(
        id="chart",
        figure=fig,
        hoverData=None,           # filled in when hovering a point
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("chart", "hoverData"))
def show(hover):
    if hover is None:                  # no point touched yet
        return "Move the mouse over a point"
    p = hover["points"][0]                 # the point under the mouse
    return f"maturity {p['x']}, spread {p['y']}"

if __name__ == "__main__":
    app.run(debug=True)
```



Hover a point: the app learns which point is under the mouse

1 the component 2 the callback that reads its value (→ moves from 1 to 2)

Try it: `python demos/components/g2_hover.py`

COMPONENT CODE

dcc.Graph · selectedData

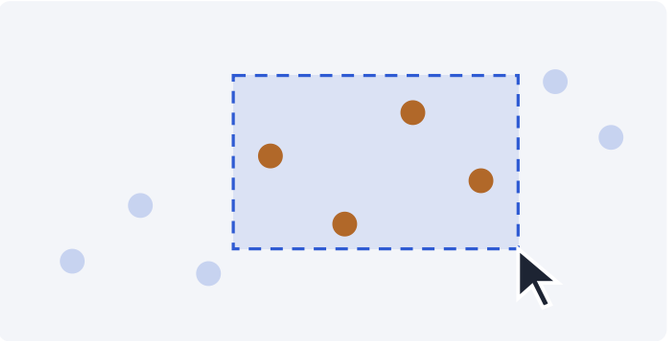
```python
"""dcc.Graph selectedData: which points are inside a box or lasso."""
from dash import Dash, html, dcc, Input, Output
import plotly.express as px

fig = px.scatter(x=[2, 3, 5, 7, 10, 20, 30],          # maturity
                 y=[80, 95, 120, 140, 165, 210, 240])  # spread
fig.update_layout(dragmode="select")   # drag draws a box, not a zoom

app = Dash(__name__)
app.layout = html.Div([
    dcc.Graph(
        id="chart",
        figure=fig,
        selectedData=None,        # filled in when a box is drawn
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("chart", "selectedData"))
def show(selected):
    if not selected:                       # nothing selected yet
        return "Drag a box around some points"
    xs = [p["x"] for p in selected["points"]]   # every point inside
    return f"{len(xs)} points, maturities {xs}"

if __name__ == "__main__":
    app.run(debug=True)
```



Draw a box or lasso: the app learns which points are inside

1 the component 2 the callback that reads its value
(→ moves from 1 to 2)

Try it: `python demos/components/g3_select.py`

COMPONENT CODE

dcc.Graph · relayoutData

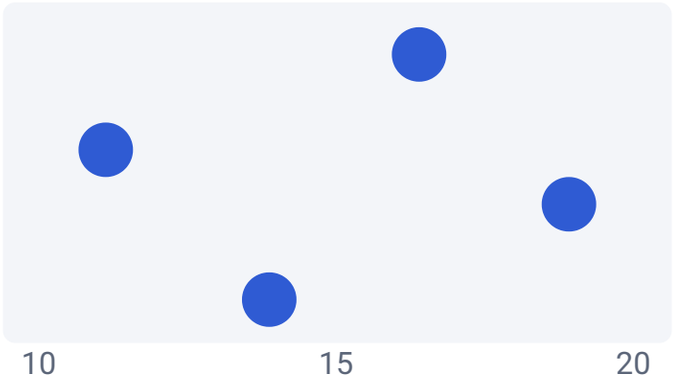
```python
"""dcc.Graph relayoutData: the visible range after a zoom or pan."""
from dash import Dash, html, dcc, Input, Output
import plotly.express as px

fig = px.scatter(x=[2, 3, 5, 7, 10, 20, 30],          # maturity
                 y=[80, 95, 120, 140, 165, 210, 240])  # spread

app = Dash(__name__)
app.layout = html.Div([
    dcc.Graph(
        id="chart",
        figure=fig,
        relayoutData=None,        # filled in on every zoom or pan
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("chart", "relayoutData"))
def show(layout):
    if not layout or "xaxis.range[0]" not in layout:  # no zoom yet
        return "Drag a box to zoom in"
    lo, hi = layout["xaxis.range[0]"], layout["xaxis.range[1]"]
    return f"showing maturities {lo:.1f} to {hi:.1f}"

if __name__ == "__main__":
    app.run(debug=True)
```



Zoom or pan: the app learns the new visible range

1 the component 2 the callback that reads its value (→ moves from 1 to 2)

Try it: `python demos/components/g4_relayout.py`

COMPONENT CODE

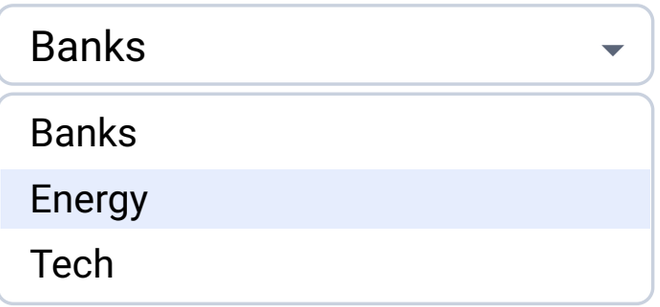
dcc.Dropdown

```python
"""dcc.Dropdown: pick from a list; the app gets the chosen item."""
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.Dropdown(
        id="sector",
        options=["Banks", "Energy", "Tech"],
        value="Banks",                    # chosen at the start
        # multi=True,          # pick several: value becomes a list
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("sector", "value"))
def show(sector):
    return f"sector = {sector!r}"

if __name__ == "__main__":
    app.run(debug=True)
```



Pick from a list: the app gets the chosen item (or several)

1 the component 2 the callback that reads its value
(→ moves from 1 to 2)

`Try it: python demos/components/w1_dropdown.py`

COMPONENT CODE

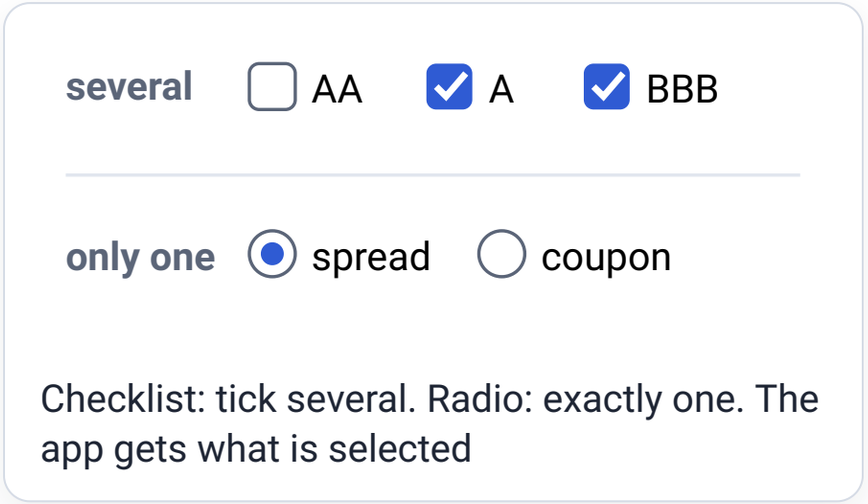
dcc.Checklist · dcc.RadioItems

```python
"""dcc.Checklist (tick several) and dcc.RadioItems (exactly one)."""
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.Checklist(
        id="ratings",
        options=["AA", "A", "BBB"],
        value=["A", "BBB"],    # a list: several can be ticked
    ),
    dcc.RadioItems(
        id="measure",
        options=["spread", "coupon"],
        value="spread",        # one value: exactly one is chosen
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"),
              Input("ratings", "value"), Input("measure", "value"))
def show(ratings, measure):
    return f"ratings = {ratings}\nmeasure = {measure!r}"

if __name__ == "__main__":
    app.run(debug=True)
```



Checklist: tick several. Radio: exactly one. The app gets what is selected

1 the component 2 the callback that reads its value (→ moves from 1 to 2)

Try it: python demos/components/w2_checklist_radio.py

COMPONENT CODE

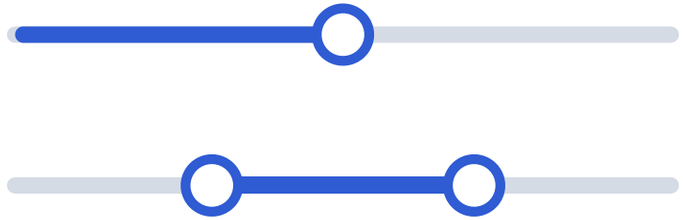
dcc.Slider · dcc.RangeSlider

```python
"""dcc.Slider (one number) and dcc.RangeSlider (a low and a high)."""
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.Slider(
        id="max_spread",
        min=0, max=400, step=10,
        value=200,                          # one number
    ),
    dcc.RangeSlider(
        id="maturity",
        min=0, max=30, step=1,
        value=[2, 10],                      # [low, high]
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"),
              Input("max_spread", "value"), Input("maturity", "value"))
def show(max_spread, maturity):
    return f"max spread = {max_spread}\nmaturity = {maturity}"

if __name__ == "__main__":
    app.run(debug=True)
```



Drag a handle: the app gets the number, or the range

1 the component 2 the callback that reads its value (→ moves from 1 to 2)

Try it: `python demos/components/w3_slider.py`

COMPONENT CODE

dcc.Input

```python
"""dcc.Input: type text or a number; the app gets what was typed."""
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.Input(
        id="issuer",
        type="text",           # or "number"
        placeholder="issuer name",
        debounce=True,         # send on Enter, not on every key
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("issuer", "value"))
def show(issuer):
    return f"issuer = {issuer!r}"

if __name__ == "__main__":
    app.run(debug=True)
```



Type text or a number: the app gets what was typed

1 the component 2 the callback that reads its value
(→ moves from 1 to 2)

Try it: `python demos/components/w4_input.py`

COMPONENT CODE

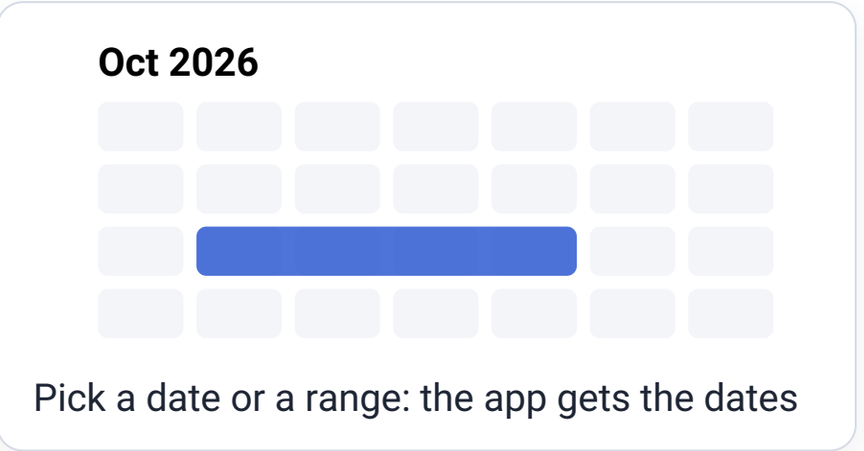
dcc.DatePickerSingle · DatePickerRange

```python
"""dcc.DatePickerSingle (one date) and dcc.DatePickerRange (two)."""
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.DatePickerSingle(
        id="asof",
        date="2026-10-27",
    ),
    dcc.DatePickerRange(
        id="window",
        start_date="2026-10-05",
        end_date="2026-10-16",
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("asof", "date"),
              Input("window", "start_date"), Input("window", "end_date"))
def show(asof, start, end):
    return f"as of {asof}\nwindow {start} to {end}"  # dates are text

if __name__ == "__main__":
    app.run(debug=True)
```



1 the component 2 the callback that reads its value
(→ moves from 1 to 2)

Try it: python demos/components/w5_dates.py

COMPONENT CODE

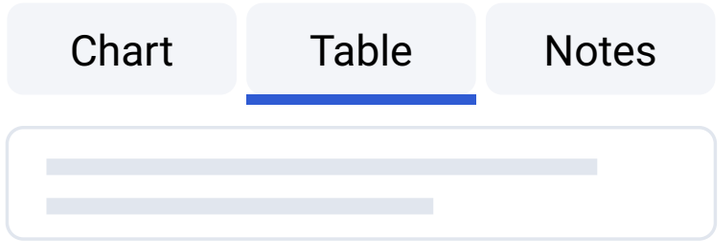
dcc.Tabs

```python
"""dcc.Tabs: the app gets which tab is open."""
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.Tabs(id="view", value="chart", children=[
        dcc.Tab(label="Chart", value="chart"),
        dcc.Tab(label="Table", value="table"),
        dcc.Tab(label="Notes", value="notes"),
    ]),
    html.Div(id="page"),
])

@app.callback(Output("page", "children"), Input("view", "value"))
def show(tab):
    return html.P(f"Showing the {tab} tab")  # what the tab shows

if __name__ == "__main__":
    app.run(debug=True)
```



Switch tabs: the app gets which tab is open

1 the component 2 the callback that reads its value
(→ moves from 1 to 2)

Try it: `python demos/components/w6_tabs.py`

COMPONENT CODE

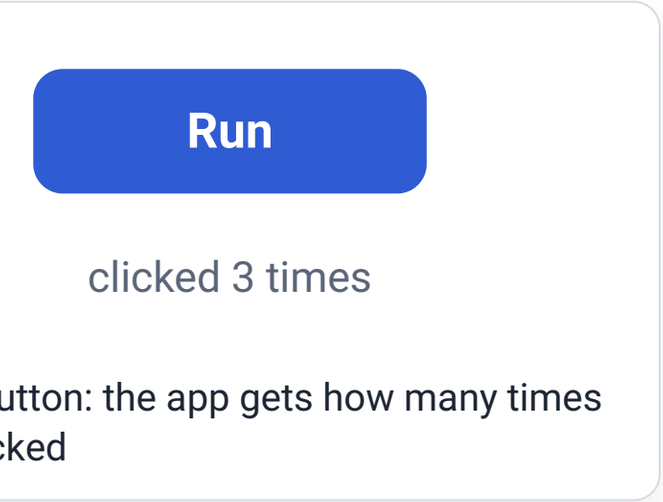
html.Button

```python
"""html.Button: the app gets how many times it was clicked."""
from dash import Dash, html, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    html.Button("Run", id="run"),          # n_clicks counts the clicks
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("run", "n_clicks"))
def show(n):
    if n is None:                          # not clicked yet
        return "Press Run"
    return f"clicked {n} time{'s' if n > 1 else ''}"

if __name__ == "__main__":
    app.run(debug=True)
```



1 the component 2 the callback that reads its value (→ moves from 1 to 2)

`Try it: python demos/components/w7_button.py`

COMPONENT CODE

dcc.Interval

```python
"""dcc.Interval: a timer; the app gets a tick every few seconds."""
from datetime import datetime
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.Interval(
        id="timer",
        interval=2000,                  # milliseconds between ticks
    ),
    html.Pre(id="out"),
])

@app.callback(Output("out", "children"), Input("timer", "n_intervals"))
def refresh(n):
    # re-read live data here, e.g. the latest prices
    return f"tick {n} at {datetime.now():%H:%M:%S}"

if __name__ == "__main__":
    app.run(debug=True)
```



tick

every few seconds

A timer: the app gets a tick every few seconds, for live updates

1 the component 2 the callback that reads its value
(→ moves from 1 to 2)

`Try it: python demos/components/w8_interval.py`

Next: live tick data from a database →

COMPONENT CODE

dcc.Interval · live tick data

```python
"""dcc.Interval + a tick database: fetch the new ticks, append them."""
from datetime import datetime
from dash import Dash, html, dcc, Input, Output, State, no_update
import plotly.graph_objects as go
from tickdb import ticks_since         # swap in your own DB query

app = Dash(__name__)
app.layout = html.Div([
    dcc.Graph(id="chart",
              figure=go.Figure(go.Scatter(x=[], y=[], mode="lines"))),
    dcc.Interval(id="timer", interval=1000),   # ask every second
    dcc.Store(id="last_ts", data=0),           # newest tick on screen
])

@app.callback(Output("chart", "extendData"), Output("last_ts", "data"),
              Input("timer", "n_intervals"), State("last_ts", "data"))
def add_ticks(n, last_ts):
    rows = ticks_since(last_ts)            # only the ticks not yet shown
    if not rows:
        return no_update, no_update
    ts, price = zip(*rows)
    x = [datetime.fromtimestamp(t) for t in ts]
    new = dict(x=[x], y=[list(price)])     # one list per trace
    return (new, [0], 2000), ts[-1]        # trace 0, keep last 2000

if __name__ == "__main__":
    app.run(debug=True)
```

Can it show live tick data? Yes, by asking on a timer:

1. Every second, `dcc.Interval` runs the callback.
2. It asks the database only for ticks newer than the last one on screen. `dcc.Store` remembers that time.
3. `extendData` adds the new points to the chart instead of redrawing it, and keeps the last 2,000.

> **The limit:** the browser asks; the server cannot push. Each open page sends one request a second, so the chart can be up to a second behind. Fine for prices on a desk. For every tick, faster updates or many viewers: let the database return 1-second bars or the last price, or push over a WebSocket (`dash-extensions`).

`Try it: python demos/components/w8b_ticks.py`

COMPONENT CODE

dcc.Store · dcc.Upload · dcc.Download

```python
"""dcc.Store (keep data), dcc.Upload (in), dcc.Download (out)."""
from dash import Dash, html, dcc, Input, Output

app = Dash(__name__)
app.layout = html.Div([
    dcc.Upload(html.Button("Upload a file"), id="up"),
    dcc.Store(id="kept"),              # keeps data in the browser
    html.Button("Download CSV", id="btn"),
    dcc.Download(id="down"),
    html.Pre(id="out"),
])

@app.callback(Output("kept", "data"), Input("up", "filename"),
              prevent_initial_call=True)
def keep(filename):
    return {"last upload": filename}

@app.callback(Output("out", "children"), Input("kept", "data"))
def show(kept):
    return f"stored: {kept}"

@app.callback(Output("down", "data"), Input("btn", "n_clicks"),
              prevent_initial_call=True)
def download(n):
    return dcc.send_string("cusip,spread\nNOR05123,142\n", "bonds.csv")

if __name__ == "__main__":
    app.run(debug=True)
```



Keep data between steps; let files in and out

1 the component 2 the callback that reads its value (→ moves from 1 to 2)

`Try it: python demos/components/w9_store_files.py`